

Problem Set 6 – DNA Sequence Marker Finder

Kshitiz Reddy
M.Tech AIML, School of AI, Bennett University

Source Code

```
1  # "Call by object reference" means the parameter name inside a function
2  # gets bound to the same object the caller's variable points to - not a
3  # copy (that's pass by value, like Java does with primitives) and not a
4  # pointer-to-a-pointer either (that's pass by reference, like C++ &args).
5  # It's just two names pointing at one object. If the function mutates
6  # that object (like .append()), the change shows up everywhere, because
7  # there was only ever one object. If the function reassigns the local
8  # name to something else, that only repoints the local name - the
9  # caller's variable still points at the original object.
10 #
11 # Author: Kshitiz
12
13
14 def search4letters(phrase: str, letters: str = 'ATCG') -> set:
15     """Return the set of target letters found in a supplied phrase."""
16     return set(letters).intersection(set(phrase))
17
18
19 def analyze_sequence(sequence_list, marker):
20     print(f"Inside Function (Start) - ID: {id(sequence_list)} | "
21           f"Data: {sequence_list}")
22     sequence_list.append(marker)
23     print(f"Inside Function (End) - ID: {id(sequence_list)} | "
24           f"Data: {sequence_list}")
25
26
27 def analyze_sequence_rebind(sequence_list, marker):
28     print(f"Inside (Start) - ID: {id(sequence_list)}")
29     sequence_list = sequence_list + [marker]
30     print(f"Inside (End) - ID: {id(sequence_list)} | Data: {sequence_list}")
31
32
33 # --- Task 1: search4letters ---
34 print("--- Task 1: search4letters ---")
35 print("Positional:", search4letters('TGGACC', 'GC'))
36 print("Keyword:", search4letters(letters='CG', phrase='TGGACC'))
37 print("Default markers:", search4letters('TGGACC'))
38
39 # --- Task 2: mutation demo ---
40 print()
41 print("--- Task 2: Mutation Demo ---")
42 dna_database = ['AATCCG', 'TGGCTA']
43 print(f"Global Scope (Before) - ID: {id(dna_database)} | Data: {dna_database}")
44 analyze_sequence(dna_database, 'CGAT')
45 print(f"Global Scope (After) - ID: {id(dna_database)} | Data: {dna_database}")
46
47 # --- Slice experiment ---
48 print()
49 print("--- Slice Experiment ---")
50 dna_database = ['AATCCG', 'TGGCTA']
51 print(f"Global Scope (Before) - ID: {id(dna_database)} | Data: {dna_database}")
```

```

52 analyze_sequence(dna_database[:], 'CGAT')
53 print(f"Global Scope (After) - ID: {id(dna_database)} | Data: {dna_database}")
54
55 # --- Bonus: rebind experiment ---
56 print()
57 print("--- Bonus: Rebind Experiment ---")
58 dna_database = ['AATCCG', 'TGGCTA']
59 print(f"Global Scope (Before) - ID: {id(dna_database)} | Data: {dna_database}")
60 analyze_sequence_rebind(dna_database, 'CGAT')
61 print(f"Global Scope (After) - ID: {id(dna_database)} | Data: {dna_database}")
62
63 # Submission Note (Discovery Challenge answer, written before checking
64 # the provided explanation): dna_database got modified even though
65 # analyze_sequence never returned anything because the function's
66 # parameter and the global variable point at the same list object - the
67 # function was never given a copy. .append() edits that object in place,
68 # so both names see the change, no return needed. My prediction matches
69 # the explanation given in the assignment.

```

Trace Log

```

1  --- Task 1: search4letters ---
2  Positional: {'C', 'G'}
3  Keyword: {'C', 'G'}
4  Default markers: {'T', 'C', 'G', 'A'}
5
6  --- Task 2: Mutation Demo ---
7  Global Scope (Before) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA']
8  Inside Function (Start) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA']
9  Inside Function (End) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA', '
   CGAT']
10 Global Scope (After) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA', '
   CGAT']
11
12 --- Slice Experiment ---
13 Global Scope (Before) - ID: 140606900748224 | Data: ['AATCCG', 'TGGCTA']
14 Inside Function (Start) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA']
15 Inside Function (End) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA', '
   CGAT']
16 Global Scope (After) - ID: 140606900748224 | Data: ['AATCCG', 'TGGCTA']
17
18 --- Bonus: Rebind Experiment ---
19 Global Scope (Before) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA']
20 Inside (Start) - ID: 140606902555264
21 Inside (End) - ID: 140606900677952 | Data: ['AATCCG', 'TGGCTA', 'CGAT']
22 Global Scope (After) - ID: 140606902555264 | Data: ['AATCCG', 'TGGCTA']

```

Task 1 results match the assignment's table exactly. In the Mutation Demo all three IDs are identical and CGAT is visible globally. In the Slice Experiment the inside-function ID differs from the global ID and the original list is untouched. In the Bonus Experiment the start ID matches the global ID but the end ID does not, and the global list stays unchanged (actual numeric IDs will differ run to run – only the pattern of sameness matters).

Submission Note

`dna_database` got modified even though `analyze_sequence` never returned anything because the function's parameter and the global variable point at the same list object – the function was never given a copy. `.append()` edits that object in place, so both names see the

change, no return needed. This prediction matches the explanation given in the assignment.